

Programação Orientada a Fluxo

1. Um fluxo de execução é composto por dado e função;



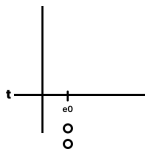
```
const a;
```

```
function f(context){}
```

2. Um dado é uma unidade de informação;

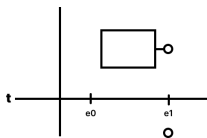


3. Um estado pode possuir **n** dados;



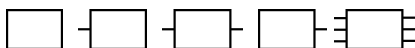
```
const a, b;
```

4. Uma função é uma unidade de mudança de estado;



```
function f(context){  
  return "hello world";  
}
```

5. Uma função pode possuir **n** endereços de entrada e **n** endereços de saídas;



```
function f(context) { }
```

```
function f(context) {
```

```

const { a } = context;
}

function f(context) {
  const { a } = context;

  return a;
}

function f(context) {
  return;
}

function f(context) {
  const { a, b, c } = context;

  return { a, b, c };
}

```

6. Funções e dados podem possuir endereços concretos;



7. Funções e dados podem possuir endereços abstratos;



8. Uma função pode receber endereços abstratos de dado e função como entrada;



```

function f(context) {
  const { a } = context;
}

function f(context) {
  const { f1 } = context;
}

```

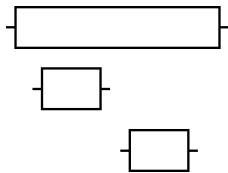
9. Uma função pode retornar endereços concretos de dado e função como saída;



```
function f(context) {
  return "";
}

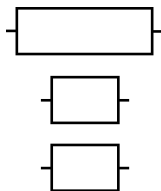
function f(context) {
  return function () { }
}
```

10. Funções podem possuir outras funções sequenciais;



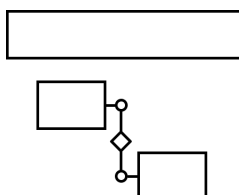
```
async function f(context) {
  await f1({ ...context });
  await f2({ ...context });
}
```

11. Funções podem possuir outras funções paralelas;



```
async function f(context) {
  await Promise.all([
    f1({ ...context }),
    f2({ ...context })
  ]);
}
```

12. Funções podem possuir estruturas de seleção;



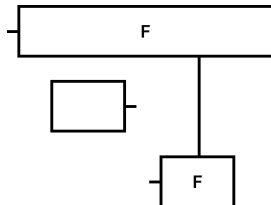
```

async function f(context) {
  const a = await f1({ ...context });

  if (a) {
    await f2({ ...context, a });
  }
}

```

13. Funções podem ter estruturas de repetição;



```

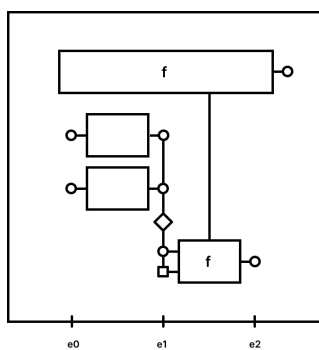
async function f(context) {
  const { a } = context;

  const a = await f1({ ...context });

  await f({ ...context, a });
}

```

14. Endereços que conhecem um ao outro fazem parte da mesma camada;



```

import f1 from './f1';
import f2 from './f2';
import a from './a';

export default async function f(context) {
  const [b, c] = await Promise.all([
    f1({ ...context, a }),
    f2({ ...context, a }),
  ]);
}

```

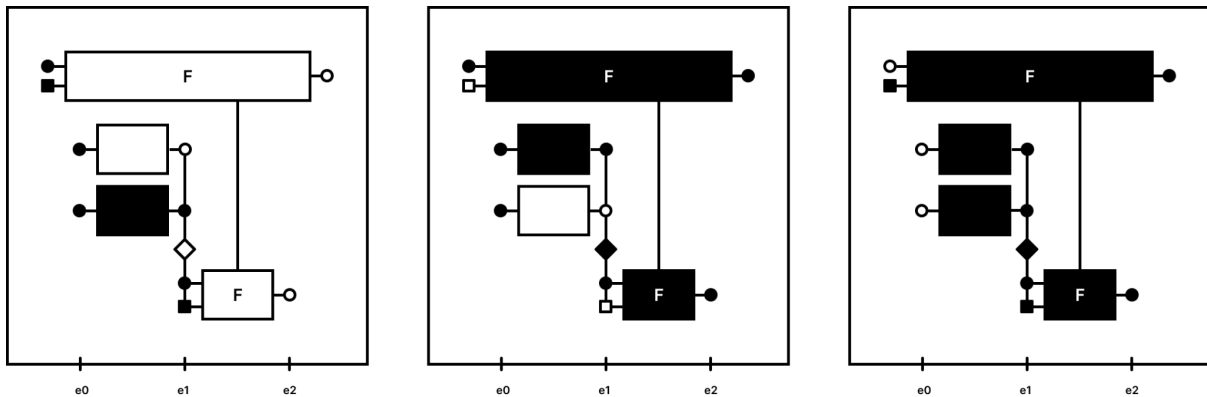
```

if (b && c) {
  return f({ ...context, a });
}

return c;
}

```

15. Endereços que não conhecem um ao outro estão em diferentes camadas;



```

import f1 from './f1';

export default async function f(context) {
  const { a, f2 } = context;

  const [b, c] = await Promise.all([
    f1({ ...context, a }),
    f2({ ...context, a }),
  ]);

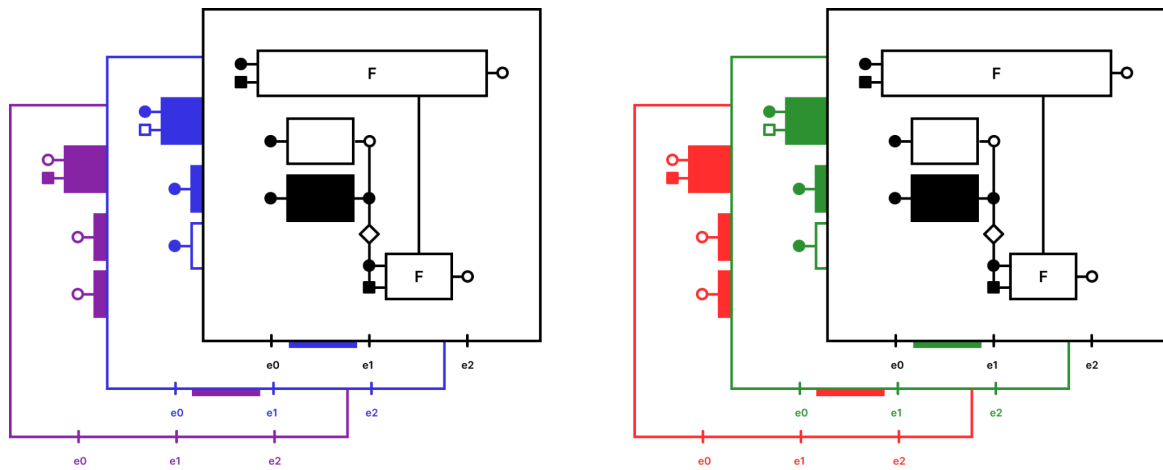
  if (b && c) {
    return f({ ...context, a, f2 });
  }

  return c;
}

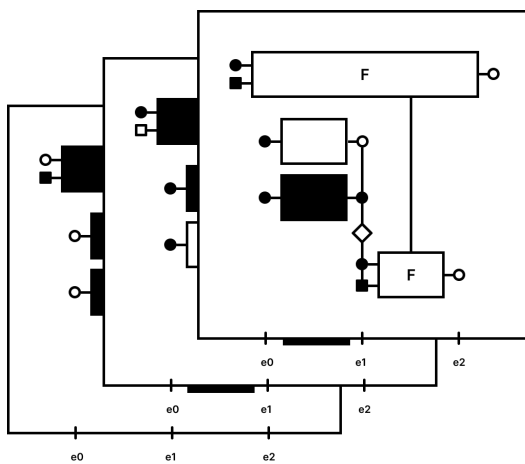
```

16. Toda entrada forma uma camada;

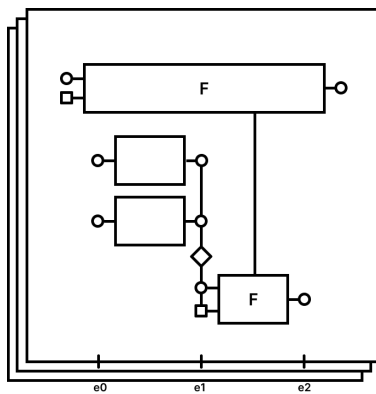
17. Uma camada é uma unidade substituível;



18. Diferentes camadas formam um contexto;



19. A execução do fluxo ocorre quando o contexto está completo;



```
import f1 from './f1';
```

```
await f({
  a: '',
  f2: () => {}
});

export default async function f(context) {
  const { a, f2 } = context;

  const [b, c] = await Promise.all([
    f1({ ...context, a }),
    f2({ ...context, a }),
  ]);

  if (b && c) {
    return f({ ...context, a, f2 });
  }

  return c;
}
```